

AgentCare

Multi-Agent Healthcare Administration System

Comprehensive Project Documentation

Prepared for AgentCare Build Challenge 2026

1. Executive Summary

AgentCare is an AI-powered healthcare administration platform designed to automate non-clinical hospital workflows using a multi-agent architecture. The system focuses exclusively on administrative processes such as appointment scheduling, patient routing, document management, follow-up coordination, and workflow auditing. It deliberately avoids diagnosis or treatment recommendations by incorporating a dedicated Safety Agent that escalates medical advice requests to human professionals.

The solution combines FastAPI, LangGraph, LangChain, SQLAlchemy, SQLite, JWT authentication, and Groq LLMs into a modular architecture that is maintainable, scalable, and production-oriented.

Implementation details emphasize modularity, clean architecture, scalability, maintainability, error handling, dependency injection, database persistence, and reusable agent tools. Each component communicates through a shared workflow state, enabling traceability, extensibility, and reliable orchestration across the complete administrative lifecycle.

2. Problem Statement

Healthcare organizations spend considerable effort handling repetitive administrative activities. Patients frequently call hospitals to book appointments, upload reports, determine the correct department, or request follow-up information. These tasks consume staff time and are often spread across disconnected systems. AgentCare centralizes these processes into a single AI-driven workflow with persistent state and auditability.

Implementation details emphasize modularity, clean architecture, scalability, maintainability, error handling, dependency injection, database persistence, and reusable agent tools. Each component communicates through a shared workflow state, enabling traceability, extensibility, and reliable orchestration across the complete administrative lifecycle.

3. Objectives

- Automate administrative workflows.
- Provide secure authentication and RBAC.
- Use specialized AI agents instead of a single monolithic model.
- Maintain persistent workflow state.
- Integrate real backend tools and databases.
- Ensure every workflow is audited.
- Prevent unsafe medical advice through human escalation.

Implementation details emphasize modularity, clean architecture, scalability, maintainability, error handling, dependency injection, database persistence, and reusable agent tools. Each component communicates through a shared workflow state, enabling traceability, extensibility, and reliable orchestration across the complete administrative lifecycle.

4. Overall Architecture

The application follows a layered architecture consisting of Presentation Layer (FastAPI + Bootstrap UI), API Layer, Service Layer, Repository Layer, Database Layer, Tool Layer, and LangGraph Workflow Layer. Requests enter through FastAPI, are authenticated, executed by the workflow graph, persisted using SQLAlchemy repositories, and returned as structured responses.

Implementation details emphasize modularity, clean architecture, scalability, maintainability, error handling, dependency injection, database persistence, and reusable agent tools. Each component communicates through a shared workflow state, enabling traceability, extensibility, and reliable orchestration across the complete administrative lifecycle.

5. Multi-Agent Design

Coordinator Agent understands intent and builds the execution plan.

Routing Agent selects the appropriate hospital department.

Appointment Agent books, reschedules, or cancels appointments.

Document Agent validates uploads, detects duplicates using SHA-256, extracts text, and classifies documents.

Follow-up Agent creates reminders and workflow summaries.

Safety Agent validates outputs, blocks medical advice, creates escalations, and records safety audits.

Implementation details emphasize modularity, clean architecture, scalability, maintainability, error handling, dependency injection, database persistence, and reusable agent tools. Each component communicates through a shared workflow state, enabling traceability, extensibility, and reliable orchestration across the complete administrative lifecycle.

6. Workflow Lifecycle

1. User authenticates.
2. Coordinator identifies intent.
3. Routing determines department when required.
4. Appointment or Document agent executes business logic.
5. Follow-up creates reminders or summaries.
6. Safety validates the workflow.
7. Final state is persisted and returned.

Implementation details emphasize modularity, clean architecture, scalability, maintainability, error handling, dependency injection, database persistence, and reusable agent tools. Each component communicates through a shared workflow state, enabling traceability, extensibility, and reliable orchestration across the complete administrative lifecycle.

7. Database Design

Key entities:

- Users
- Patient Profiles
- Departments
- Doctors
- Appointment Slots
- Appointments
- Patient Documents
- Workflow Runs
- Reminders
- Escalations
- Audit Events

Relationships enforce referential integrity while repositories encapsulate CRUD operations.

Implementation details emphasize modularity, clean architecture, scalability, maintainability, error handling, dependency injection, database persistence, and reusable agent tools. Each component communicates through a shared workflow state, enabling traceability, extensibility, and reliable orchestration across the complete administrative lifecycle.

8. Security

JWT-based authentication protects APIs. Role-Based Access Control restricts endpoints for patients, staff, and administrators. Passwords are securely hashed with bcrypt. Every sensitive operation generates an audit event. Duplicate document detection relies on SHA-256 checksums. Medical requests are escalated instead of answered.

Implementation details emphasize modularity, clean architecture, scalability, maintainability, error handling, dependency injection, database persistence, and reusable agent tools. Each component communicates through a shared workflow state, enabling traceability, extensibility, and reliable orchestration across the complete administrative lifecycle.

9. Testing Strategy

The project contains unit and integration tests covering authentication, repositories, services, REST APIs, and LangGraph agents. LLM interactions are mocked where appropriate, allowing deterministic testing without external API calls.

Implementation details emphasize modularity, clean architecture, scalability, maintainability, error handling, dependency injection, database persistence, and reusable agent tools. Each component communicates through a shared workflow state, enabling traceability, extensibility, and reliable orchestration across the complete administrative lifecycle.

10. Deployment

Create a Python virtual environment, install dependencies, configure .env variables, run Alembic migrations, seed the database, execute pytest, and launch the application with Uvicorn. SQLite provides persistence while ChromaDB may be used for vector storage.

Implementation details emphasize modularity, clean architecture, scalability, maintainability, error handling, dependency injection, database persistence, and reusable agent tools. Each component communicates through a shared workflow state, enabling traceability, extensibility, and reliable orchestration across the complete administrative lifecycle.

11. Challenges and Design Decisions

Important engineering decisions included separating orchestration from business logic, implementing repository and service patterns, ensuring every workflow step is auditable, enabling deterministic testing through dependency injection and mocking, and keeping safety isolated from task execution.

Implementation details emphasize modularity, clean architecture, scalability, maintainability, error handling, dependency injection, database persistence, and reusable agent tools. Each component communicates through a shared workflow state, enabling traceability, extensibility, and reliable orchestration across the complete administrative lifecycle.

12. Future Enhancements

Future work includes OCR improvements, email/SMS reminders, cloud deployment, analytics dashboards, multi-tenancy, WebSocket notifications, calendar integrations, multilingual support, and additional hospital information system integrations.

Implementation details emphasize modularity, clean architecture, scalability, maintainability, error handling, dependency injection, database persistence, and reusable agent tools. Each component communicates through a shared workflow state, enabling traceability, extensibility, and reliable orchestration across the complete administrative lifecycle.

13. Conclusion

AgentCare demonstrates how specialized AI agents can safely automate healthcare administration. Its modular architecture, persistent workflow state, secure authentication, audit logging, and human-in-the-loop safety model make it a strong foundation for enterprise healthcare administration systems.

Implementation details emphasize modularity, clean architecture, scalability, maintainability, error handling, dependency injection, database persistence, and reusable agent tools. Each component communicates through a shared workflow state, enabling traceability, extensibility, and reliable orchestration across the complete administrative lifecycle.